

LAB 8

DINING PHILOSOPHER PROBLEM

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_PHILOSOPHERS 5

pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];

void* philosopher(void* arg) {    int id = *(int*)arg;
    int left_chopstick = id;
        int right_chopstick = (id + 1) % NUM_PHILOSOPHERS;
    for (int i = 0; i < 2; i++) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(rand() % 2 + 1);        if (id % 2 == 0) {
            pthread_mutex_lock(&chopsticks[left_chopstick]);
        pthread_mutex_lock(&chopsticks[right_chopstick]);
            } else {
                pthread_mutex_lock(&chopsticks[right_chopstick]);
        pthread_mutex_lock(&chopsticks[left_chopstick]);
            }
        printf("Philosopher %d picked up chopsticks %d and %d and is EATING.\n", id,
        left_chopstick, right_chopstick);        sleep(rand() % 2 + 1);
            pthread_mutex_unlock(&chopsticks[left_chopstick]);
        pthread_mutex_unlock(&chopsticks[right_chopstick]);
            printf("Philosopher %d finished eating and put down chopsticks.\n", id);
        }
        return NULL;
    }

int main() {
```

```

    pthread_t philosophers[NUM_PHILOSOPHERS];
int ids[NUM_PHILOSOPHERS];
srand(time(NULL));

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {        if
(pthread_mutex_init(&chopsticks[i], NULL) != 0) {
printf("Mutex initialization failed.\n");        return 1;
    }
}

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
ids[i] = i;

        if (pthread_create(&philosophers[i], NULL, philosopher, &ids[i]) != 0) {
printf("Thread creation failed.\n");        return 1;
        }
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
pthread_join(philosophers[i], NULL);
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
pthread_mutex_destroy(&chopsticks[i]);
    }

    printf("All philosophers have finished dining.\n");
return 0;
}

```

```
C:\Users\student\Downloads\ X + v
Philosopher 4 picked up chopsticks 4 and 0 and is EATING.
Philosopher 2 picked up chopsticks 2 and 3 and is EATING.
Philosopher 0 picked up chopsticks 0 and 1 and is EATING.
Philosopher 4 finished eating and put down chopsticks.
Philosopher 4 is thinking.
Philosopher 3 picked up chopsticks 3 and 4 and is EATING.
Philosopher 2 finished eating and put down chopsticks.
Philosopher 2 is thinking.
Philosopher 1 picked up chopsticks 1 and 2 and is EATING.
Philosopher 0 finished eating and put down chopsticks.
Philosopher 0 is thinking.
Philosopher 4 picked up chopsticks 4 and 0 and is EATING.
Philosopher 3 finished eating and put down chopsticks.
Philosopher 3 is thinking.
Philosopher 4 finished eating and put down chopsticks.
Philosopher 3 picked up chopsticks 3 and 4 and is EATING.
Philosopher 0 picked up chopsticks 0 and 1 and is EATING.
Philosopher 1 finished eating and put down chopsticks.
Philosopher 1 is thinking.
Philosopher 2 picked up chopsticks 2 and 3 and is EATING.
Philosopher 3 finished eating and put down chopsticks.
Philosopher 2 finished eating and put down chopsticks.
Philosopher 0 finished eating and put down chopsticks.
Philosopher 1 picked up chopsticks 1 and 2 and is EATING.
Philosopher 1 finished eating and put down chopsticks.
All philosophers have finished dining.

Process returned 0 (0x0)   execution time : 10.203 s
Press any key to continue.
```

2. DEADLOCK DETECTION ALGORITHM

```
#include <stdio.h> #include
```

```
<stdbool.h>
```

```
void detectDeadlock(int P, int R, int processes[], int available[], int allocation[][R], int
```

```
request[][R]) {   int work[R];   bool finish[P];   int safe_sequence[P];   int
```

```
safe_count = 0;   int deadlocked_processes[P];   int deadlock_count = 0;   for (int i
```

```
= 0; i < R; i++) {       work[i] = available[i];
```

```
   }
```

```
   for (int i = 0; i < P; i++) {
```

```
bool has_allocation = false;
```

```
for (int j = 0; j < R; j++) {
```

```
if (allocation[i][j] != 0) {
```

```
has_allocation = true;
```

```
break;
```

```
   }
```

```
}
```

```

        finish[i] = !has_allocation;
if (finish[i]) {
    safe_sequence[safe_count++] = processes[i];
}
}

bool progress = true; while
(progress) {    progress = false;
for (int i = 0; i < P; i++) {    if
(!finish[i]) {    bool
can_allocate = true;    for (int j
= 0; j < R; j++) {    if
(request[i][j] > work[j]) {
can_allocate = false;
break;
    }    }    if
(can_allocate) {    for (int j =
0; j < R; j++) {    work[j] +=
allocation[i][j];
    }
finish[i] = true;
    safe_sequence[safe_count++] = processes[i];
progress = true;
    }
    }
}

for (int i = 0; i < P; i++) {
if (!finish[i]) {
    deadlocked_processes[deadlock_count++] = processes[i];
}
}

```

```

    }

    printf("\n--- RESULTS ---\n");
if (deadlock_count > 0) {
    printf("System is in a DEADLOCK state.\n");
    printf("Deadlocked processes: ");    for (int i =
0; i < deadlock_count; i++) {        printf("P%d
", deadlocked_processes[i]);
        }    printf("\n");
if (safe_count > 0) {
    printf("Processes that executed before deadlock: ");
for (int i = 0; i < safe_count; i++) {        printf("P%d
", safe_sequence[i]);
        }
    printf("\n");
    }
    } else {
        printf("System is SAFE. No deadlock detected.\n");
        printf("Execution Sequence: < ");    for (int i = 0; i <
safe_count; i++) {        printf("P%d ",
safe_sequence[i]);
        }
        printf(">\n");    }

} int main()
{    int P,
R;

    printf("Enter the number of processes: ");
    scanf("%d", &P);

    printf("Enter the number of resource types: ");
    scanf("%d", &R);    int processes[P];    for(int i
= 0; i < P; i++) {        processes[i] = i;

```

```

    }
    int available[R];

    printf("\nEnter the Available instances of each resource (space-separated):\n");
    for (int i = 0; i < R; i++) {        scanf("%d", &available[i]);
    }

    int allocation[P][R];

    printf("\nEnter the Allocation Matrix (%d rows, %d columns):\n", P, R);
    for (int i = 0; i < P; i++) {        for (int j = 0; j < R; j++) {
scanf("%d", &allocation[i][j]);
        }
    }

    int request[P][R];

    printf("\nEnter the Request Matrix (%d rows, %d columns):\n", P, R);
    for (int i = 0; i < P; i++) {        for (int j = 0; j < R; j++) {
scanf("%d", &request[i][j]);
        }
    }

    printf("\nRunning Deadlock Detection Algorithm...\n");
    detectDeadlock(P, R, processes, available, allocation, request);    return
0;
}
OUTPUT

```

```
C:\Users\student\Downloads\ x + v
Enter the number of processes: 5
Enter the number of resource types: 3

Enter the Available instances of each resource (space-separated):
0 0 0

Enter the Allocation Matrix (5 rows, 3 columns):
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2

Enter the Request Matrix (5 rows, 3 columns):
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2

Running Deadlock Detection Algorithm...

--- RESULTS ---
System is SAFE. No deadlock detected.
Execution Sequence: < P0 P2 P3 P4 P1 >

Process returned 0 (0x0)   execution time : 52.308 s
Press any key to continue.
|
```